基于大语言模型与启发式规则混合架构的需求歧义检测与修复实证研究

你的名字

2026 年 10 月 26 日

摘要

需求歧义是导致软件项目失败、成本超支的关键因素。传统方法在深层语义理解上存在局限，而单一的大语言模型（LLM）在专业领域易出现漏检和幻觉。本文基于刘芳（2026）提出的“多智能体混合架构”思想，设计并实现了一个缩微版的“规则预筛选 + LLM 推理”检测框架，以及基于 CoT 的“定位-解释-重写”修复框架。实验在 14 条人工标注的多类型需求数据集上进行。结果表明：纯 LLM 的检测准确率为 78.57%；引入启发式规则并优化优先级后，准确率提升至 100%。在修复阶段，LLM 能有效消除歧义，但也暴露出原论文指出的“过度规范”风险（如随意设定具体阈值）。本实验验证了混合架构在需求工程中的有效性，为后续构建全自动需求质量保障体系提供了实践依据。

1 引言

在现代软件开发中，需求规格说明书（SRS）通常采用自然语言编写。由于自然语言的多义性，需求文本极易产生歧义（Ambiguity）。据统计，需求阶段的缺陷占比高达 50% 以上。近年来，大语言模型（如 DeepSeek、GPT-4o）展现出强大的语义理解能力，但刘芳（2026）的研究指出，单一 LLM 在多阶段推理时容易遗漏局部特征（如拼写错误、模糊词），且修复建议缺乏领域知识支撑。

本文作为一项小规模实证研究，旨在复现并验证原论文的核心思想：**“启发式规则预筛选 + 大语言模型推理”**的混合检测架构，以及**“触发片段-候选解释-模板化输出”**的修复流程。

2 实验设计与方法

2.1 数据集构建

本实验构建了包含 14 条英文软件需求语句的小型数据集，涵盖了原论文定义的六类语言歧义及无歧义对照组。

- **词汇歧义**（如 bank 多义）：2 条
- **句法歧义**（如介词短语修饰不明）：2 条
- **语义歧义**（如 only 作用域不清）：2 条

- **语用歧义**（如代词指代不明）：2 条
- **语法歧义**（如词性误用缺谓语）：1 条
- **语言错误歧义**（如拼写错误、主谓不一致）：2 条
- **无歧义**（清晰的主谓宾结构）：3 条

2.2 需求歧义检测方法

2.2.1 基线方法：纯 LLM 提示工程

直接调用 DeepSeek API，采用 Few-shot 提示词。在 Prompt 中明确给出六类歧义的定义、判定边界（如“仅仅出现 the 不构成歧义”），并要求模型输出 JSON 格式的检测结果。

2.2.2 混合架构（Rule + LLM）

借鉴原论文第 4 章思路，编写 Python 启发式规则（正则表达式），设计**规则优先级机制**：

1. **高优先级**：拼写错误拦截（如匹配到 `systm` 直接判为语言错误）。
2. **次优先级**：一致性错误拦截（如正则匹配 `every...their` 判为语言错误）。
3. **中优先级**：范围限定词拦截（如匹配 `only` 判为语义歧义）。
4. **低优先级**：模糊词拦截（如匹配 `quickly` 判为词汇歧义）。

若句子命中规则，直接输出规则判断类型；若未命中，则调用 LLM 进行推理。

2.3 需求歧义修复方法

借鉴原论文第 5 章的 LMArepair 框架，设计结构化修复 Prompt，强制大模型输出包含以下字段的 JSON：

- `trigger_segment`：导致歧义的最小片段。
- `interpretations`：可能产生的两种解释。
- `suggestion`：重写后的清晰需求句。
- `reason`：修改理由。

3 实验结果与分析

3.1 检测结果对比分析

实验发现，纯 LLM 在检测具有深层逻辑的语义/语用歧义时表现优异，但在处理“模糊词”和“拼写错误”时发生了漏检。

错误案例分析（纯 LLM 基线）：

表 1: 纯 LLM 与 混合架构检测性能对比

方法	正确数	错误数	准确率
纯 LLM 基线	11	3	78.57%
混合架构 (Rule + LLM)	14	0	**100.00%**

- 第 10 句 *"The system shall process all requests quickly."*，纯 LLM 判为无歧义，漏掉了模糊词 *quickly* 造成的词汇歧义。
- 第 13 句 *"Only administrators can delete files."*，纯 LLM 判为无歧义，未意识到 *Only* 的修饰范围可能存在语义歧义。
- 第 9 句 *"Every student must submit their assignment."*，纯 LLM 判为语义歧义，未能识别出 *Every* 与复数代词 *their* 造成的语言错误。

引入规则拦截后，上述硬伤均被精确捕捉，证明了原论文引入启发式规则的必要性。

3.2 修复结果与专家评价

在 11 条歧义需求中，修复模块成功生成了建议，但也暴露出问题：

1. **成功量化模糊词**：针对第 10 句，LLM 将 *quickly* 重写为 *within 2 seconds of receipt*，将不可测试的需求变为可量化的需求。
2. **JSON 解析幻觉**：第 2 句修复失败。原因是 LLM 在 JSON 字符串中使用了嵌套的双引号未转义，导致 `json.loads()` 报错。
3. **过度规范（Over-specification）**：这正是原论文中提到的局限性。针对第 6 句拼写错误，LLM 把 *quickly* 直接改成了 *within 2 seconds*。虽然消除了歧义，但业务方原本并未要求 2 秒，这种“画蛇添足”在实际工程中可能引发争议。
4. **语言漂移**：第 13 句 *Only administrators can delete files.*，LLM 给出的建议直接变成了中文：“只有具有管理员角色的用户才能删除文件...”，破坏了原需求的英文语言一致性。

4 讨论

通过本次 14 条数据的小规模复现，我们对刘芳论文的方法有了更直观的体会：

- **混合架构是必由之路**：正如原论文指出，大语言模型（LLM）并非全能。纯 LLM 容易陷入“过度自信”的幻觉，而简单的基于正则表达式的启发式规则，能以极低的成本弥补 LLM 对“显式特征（如拼写错误）”不敏感的短板。
- **修复需要外部知识锚定**：原论文提出 LMArepair 需要结合“维基百科领域语料库”。本次实验中 LLM 随意设定“2 秒”阈值，恰好证明了 LLM 在重写需求时缺乏项目上下文和真实业务指标的约束。

- **实验局限性**：本实验仅有 14 条单句，且规则是由人工根据当前数据“量身定制”的（存在过拟合嫌疑）。若要在真实工业场景应用，必须像原论文一样，依靠多智能体（Agent）机制让不同专家进行投票（D-S 证据理论），以提升规则的泛化能力。

5 结论

本次小科研成功复现了原论文的核心理念。实验证明，采用“规则+LLM”的混合检测架构能将准确率从 78.57% 提升至 100%。在需求修复环节，大模型虽能给出合理的改写建议，但必须警惕其“过度规范”与“格式幻觉”问题。后续研究可进一步引入多专家 Agent 投票机制，以增强系统的鲁棒性和可解释性。